

LLM과 Chatbot을 결합한 업무 Q&A봇 구현 및 설계

이아이씨엔 기업부설연구소 최남규 전병욱
아주대학교 산업공학과 이혜은, 박재일

이메일: hans@eicn.co.kr

오픈 프로필: <https://open.kakao.com/me/hanschoi>

링크드인: <https://www.linkedin.com/in/hans-choi/>

CONTENTS

- 1. 개요
- 2. 선행연구
- 3. 제안
 - 3.1 구축 기본방향
 - 3.2 Q&A봇 구성
 - 3.3 유사문서검색 모듈
 - 3.3 연동방안
- 4. 아키텍처 및 구현
- 5. 결론

1. 서론 (1/2)

❖ LLM과 챗봇

- 1) 기존 챗봇은 제한된 표현과 규칙 통해 정형적인 답변
- 2) LLM기반 Chatbot 풍부하고 다양한 표현 가능하지만, 환각증상, 프라이빗 정보에 대한 답변 어려움 [3]
- 3) LLM기반 챗봇이 기존 챗봇 대체 기술로 각광 [1]

❖ 태스크 지향 Chatbot

- 1) 특정도메인의 업무 수행 Chatbot : 명확하고 정확한 응답 중요 [2]
- 2) 코퍼스기반 Chatbot은 답변 오인식/미인식 문제 조치시 재학습 과정 필요
- 3) RAG기반 LLM: LLM의 한계 개선, Chatbot서비스 동안 즉각적인 질문 오인식/미인식 조치 불가 [4][5]

1. 서론 (2/2)

❖ 제안: RAG기반 LLM과 Chatbot을 결합한 업무 Q&A봇 구현및 설계

- 1) 유연하고 확장 가능한 아키텍처 / 개발 방법론 / 기술스택 제안
- 2) RAG기반 Chatbot과 유사도문서검색 기반 Chatbot 연계
- 3) 디버깅모드 제공: 잘못된 답변 원인 파악 용이 ▶ 순수LLM답변, RAG검색결과, RAG+LLM, 인텐트
- 4) 재학습이 필요없는 질문 오인식/미인식에 대한 오류신고 절차 및 조치 방안

2. 선행연구 – 2.1 Chatbot (1/2)

❖ Chatbot

- 1) 개요: 이용자가 의도하는 질문에 적합한 답변이나 요구사항 처리
- 2) 인텐트: 사용자 의도, 일종의 다건의 질문 그룹, 답변은 인텐트당 1건으로 할수도 있고, 랜덤 교체도 가능
- 3) 엔티티: 개체, 도메인에 관련 단어를 의미, 매개변수 활용, 인텐트 분류를 세분화 할 수 있음

❖ Chatbot 종류 [2][6]

- 1) 규칙기반: 시나리오 Chatbot으로 주로 클릭
- 2) 검색기반: 질의시 다건 추천
- 3) 코퍼스기반: BERT와 같은 구분모델 활용 인텐트 분류, 엔티티 식별 위한 개체명 인식
- 4) LLM기반: 검색결과와 질문을 LLM에 전달하여 답변 생성하는 RAG방식이 대세

2. 선행연구 – 2.1 Chatbot (2/2)

❖ 코퍼스기반 Chatbot 문제점

- 1) 사용자 발화 의도(인텐트) 파악 위한 분류학습에 많은 비용/시간 소요 [6][7][8]
- 2) 오분류로 인해 의도파악 실패시 즉각 조치 어려움: 학습데이터 구축 어려움, 모호한 분류 재검토 이슈
- 3) 분류모델의 신뢰도 임계값 정하는 문제 , 개체명인식 기술은 분류모델보다 난이도 높고 오류율 높음 [9]

2. 선행연구 – 2.2 기술스택 (1/5)

❖ 기술스택 내역

- LLM모델 서빙 및 선정
- LLM 오케스트레이션 프레임워크
- 임베딩모델
- 벡터데이터베이스
- 웹 프레임워크
- 멀티프로세스 처리 기술
- 동시성처리 및 API호출 제한
- 데이터베이스 처리 프레임워크

2. 선행연구 – 2.2 기술스택 (1/5)

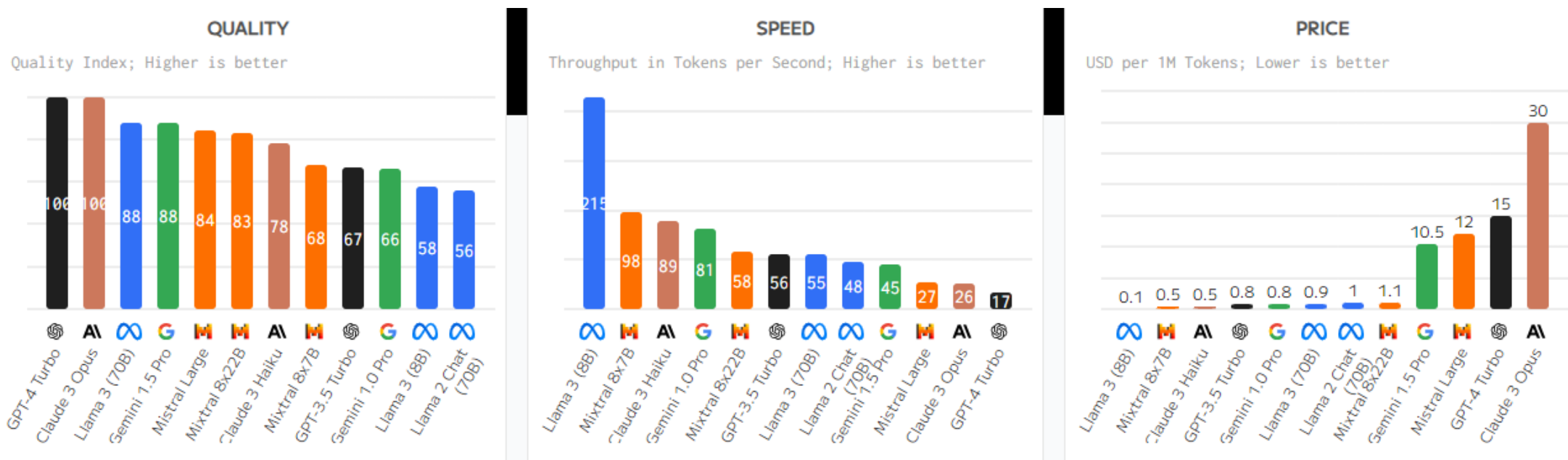
❖ LLM모델 서빙 및 선정

- 1) 클라우드 기반 LLM API 이용하거나, 온프레미스형 LLM 구축
- 2) 서비스용 모델 선정시 고려사항
 - 모델 개발한 조직이 지속적인 지원 가능한지
 - 바로 출시된 모델보다는 안정화 및 검증된 모델
 - 클라우드기반 LLM모델: 비용, 속도, 성능 고려
 - 오픈소스 LLM
 - . 한국어처리를 위한 모델 Aliment 필요한 지 점검 필요
 - . 예산범위 내의 인프라 활용 점검: 양자화/경량화 필요자원 점검, 추론/학습 필요자원 점검 필요

2. 선행연구 - 2.2 기술스택 (2/5)

❖ 벤치마크 : 비용, 속도, 성능

- 1) 가성비 좋은 클라우드 LLM: openAI의 GPT3.5 Turbo, 구글의 Gemini 1.0 Pro, Claude 3 Haiku
- 2) 가성비 좋은 오픈소스 LLM: Llama3(8B), Mixtral 8X7B, Llama 2 Chat(70B)



2. 선행연구 – 2.2 기술스택 (3/5)

❖ 한국어 특화 LLM 리더보드

- 1) Open ko-LLM leaderboard (hosted by upstage , NIA)
 - <https://huggingface.co/spaces/upstage/open-ko-llm-leaderboard>
- 2) LogicKor (hosted by instruct.kr)
 - <https://lk.instruct.kr/> (구현소스 및 설명: <https://github.com/instructkr/LogicKor>)
- 3) 금융 전문 LLM 리더보드 (hosted by 올거나이즈, <https://www.allganize.ai/>)
 - <https://llm-leaderboard.org/>
- 4) Horangi 한국어 LLM 리더보드 (hosted by wanda)
 - <https://wandb.ai/wandb-korea/korean-llm-leaderboard/reportlist>

❖ 한국어에 특화되지 않은 LLM 리더보드

- 1) Open LLM Leaderboard (hosted by HuggingFaceH4)
 - https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard
- 2) LMSYS Chatbot Arena Leaderboard (hosted by LMSYS, <https://lmsys.org/>) - 블라인드 테스트 방식
 - <https://chat.lmsys.org/?leaderboard>

2. 선행연구 – 2.2 기술스택 (4/5)

❖ 임베딩 개요

- 1) 텍스트 데이터를 머신러닝 알고리즘으로 처리할 수 있는 숫자 벡터로 변환하는 NLP 기술
- 2) 벡터로 변환된 정보를 통해 단어/문장 간 관련도의 계산 가능
- 3) 임베딩 된 정보가 벡터 저장소에 저장되어 있어 텍스트의 의미와 관련된 처리 수행

❖ 임베딩모델 추천

- 1) OpenAI에서 제공되는 text-embedding-3모델 많이 활용
 - 계속 사용시 API 이용 요금 발생
- 2) 오픈소스 임베딩 모델: baai/bge-m3 : 다국어 임베딩 처리 우수 [14]
 - 허깅페이스: <https://huggingface.co/BAAI/bge-m3>
 - 논문에서 다른 논문들과 베치마킹 테스트 수행
 - 해당 논문에서 비교군으로 사용된 임베딩모델 도입 고려
 - . BM25, mDPR, mContriever, mE5_{large}, E5_{mistral-7b}, OpenAI-3

2. 선행연구 – 2.2 기술스택 (5/5)

❖ 벡터 데이터베이스

- 1) 환각 줄이고 정확하게 대답 생성 – 태스크 지향 Chatbot이 목적에 부합
- 2) 대상 자료를 사전에 벡터DB에 저장하고, 질문 들어오면 유사도 기반 검색하여 LLM에 전달
- 3) 크로마DB, Milvus, 엘라스틱서치, 파인콘 등을 많이 사용

❖ LLM 오케스트레이션 프레임워크

- 1) 임베딩된 데이터를 벡터DB에 저장하고 해당 내용을 LLM에 전달하는 전반적인 처리
- 2) 모델이 오케스트레이션 지원안하는 경우, 해당 LLM 가이드에 따라 직접 개발
- 3) Langchain 커뮤니티가 가장 활성화, Semantic Kernel, LlamaIndex도 있음.

❖ 웹 프레임워크

- 1) Django, Flask, FastAPI 많이 사용되며, FastAPI가 고성능 API, 비동기 프로그래밍 지원으로 인기
- 2) FastAPI의 Pydantic라이브러리이용 swagger이용, 별도 코딩없이 API명세서 및 테스트 지원
- 3) OpenAPI는 Restful API를 정의된 규칙에 맞게 API 스펙을 JSON이나 yaml로 표현, Open API와 다름.

2. 선행연구 - 2.2 기술스택 (6/5)

❖ 멀티프로세싱

- 1) 다중접속자에 대한 답변 처리 중요
- 2) 파이썬의 socket과 threading 라이브러리 이용 멀티 쓰레드 생성 요청과 TCP소켓서버 연계
 - API서버와 TCP소켓서버 구성
- 3) Uvicorn과 FastAPI 통한 비동기 싱글 프로세서 API서버에 Gunicorn 연동 통해 멀티프로세스 기능 구현

❖ 동시성제어

- 1) 클라우드 기반 LLM API서비스는 보통 API호출제한(rate limit) , 초과시 에러 반환
- 2) 에러방지 위해 비동기화 기술 중 세마포어 기술 활용하여 동시 요청수 제어
- 3) 세마포어는 멀티프로그래밍 환경에서 공유자원 접근 제한, Python의 asyncio.Semaphore(n) 활용

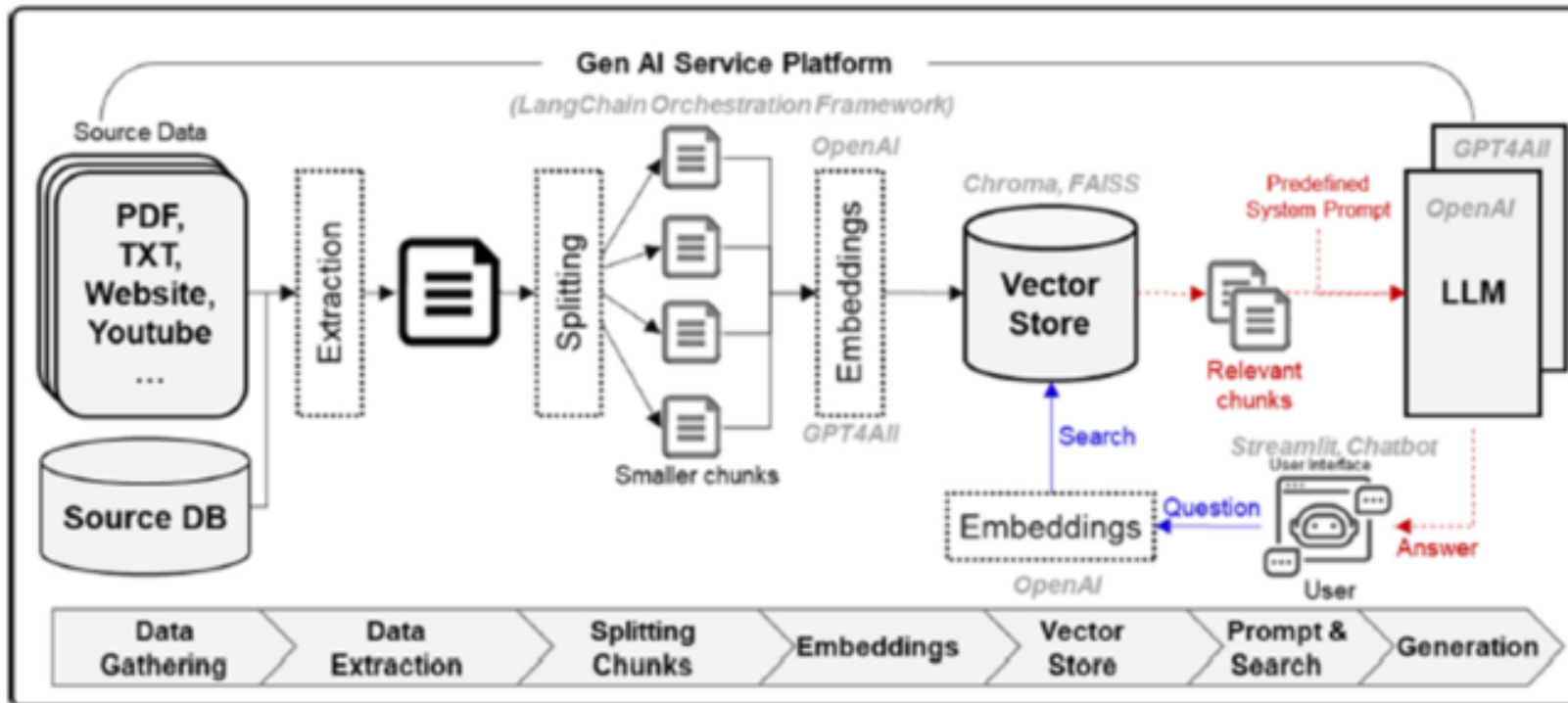
❖ 데이터베이스 처리

- 1) FastAPI의 Pydantic과 잘맞는 SQL 툴킷으로 SQLAlchemy 활용, SQL방식은 sqlite, pymysql 많이 활용
- 2) SQLAlchemy은 ORM(Object Relational Mapper) 개념 활용하여 객체와 데이터베이스 간의 매핑 간편

2. 선행연구

2.3 검색증강 생성 LLM기반 처리 프로세스

- 1) RAG기반 LLM 처리 절차는 RAG대상 문서 저장과 LLM추론시 RAG검색결과를 받아 Chatbot으로 답변
- 2) 질의 하기전에 데이터(웹, 파일, 텍스트 등) 수집하여 벡터DB에 저장하는 과정 필요
- 3) Chatbot에 사용자의 질문이 입력될때 벡터DB를 검색하여 질문과 검색결과를 LLM에 전달하여 응답



* LLM과 RAG기반 서비스 처리 프로세스 [1]

2. 선행연구

2.4 LLM 환각증상 개선

- 1) LLM의 매개변수에 질문 관련 지식없거나 학습시 데이터가 없는 경우 정확한 추론 어려운 한계
- 2) 모델 튜닝이나 모델 재학습 없어도 문맥학습 통해 정확한 답변 유도 가능
- 3) 컨텍스트 내용 많으면 비효율적

- 특히, 클라우드기반 LLM API는 토큰 또는 글자수에 따라 과금 방식으로 비용 고려

- 4) RAG 통해 사전 검색후 검색결과와 질문을 LLM 전달하여 환각증상 개선

※ 관리자화면에서 디버깅용 chatbot을 제공하여 단계별 결과 확인 가능

- 순수LLM답변, 벡터DB 검색결과, LLM & RAG 연계 답변

※ 사용자가 chatbot의 답변에 대해 오류 신고하고, 관리자가 이를 쉽게 조치할 수 있는 프로세스 제안

3. 제안

3.1 구축 기본방향

- 1) 시나리오 구축단계의 RAG 튜닝 / 개선 연구는 활발하나, 오답변/미답변에 대한 신속한 보정 연구 부족
- 2) 기존 분류모델 기반의 인텐트 식별방식은 많은 구축 비용과 시간 소요
- 3) 학습이 불필요한 유사문서 검색 기술을 통한 인텐트 식별 방안 제안
- 4) LLM의 오인식 또는 미인식 답변을 유사도기반 Chatbot 연계 통해 신속한 오류신고 및 조치 방안 제시

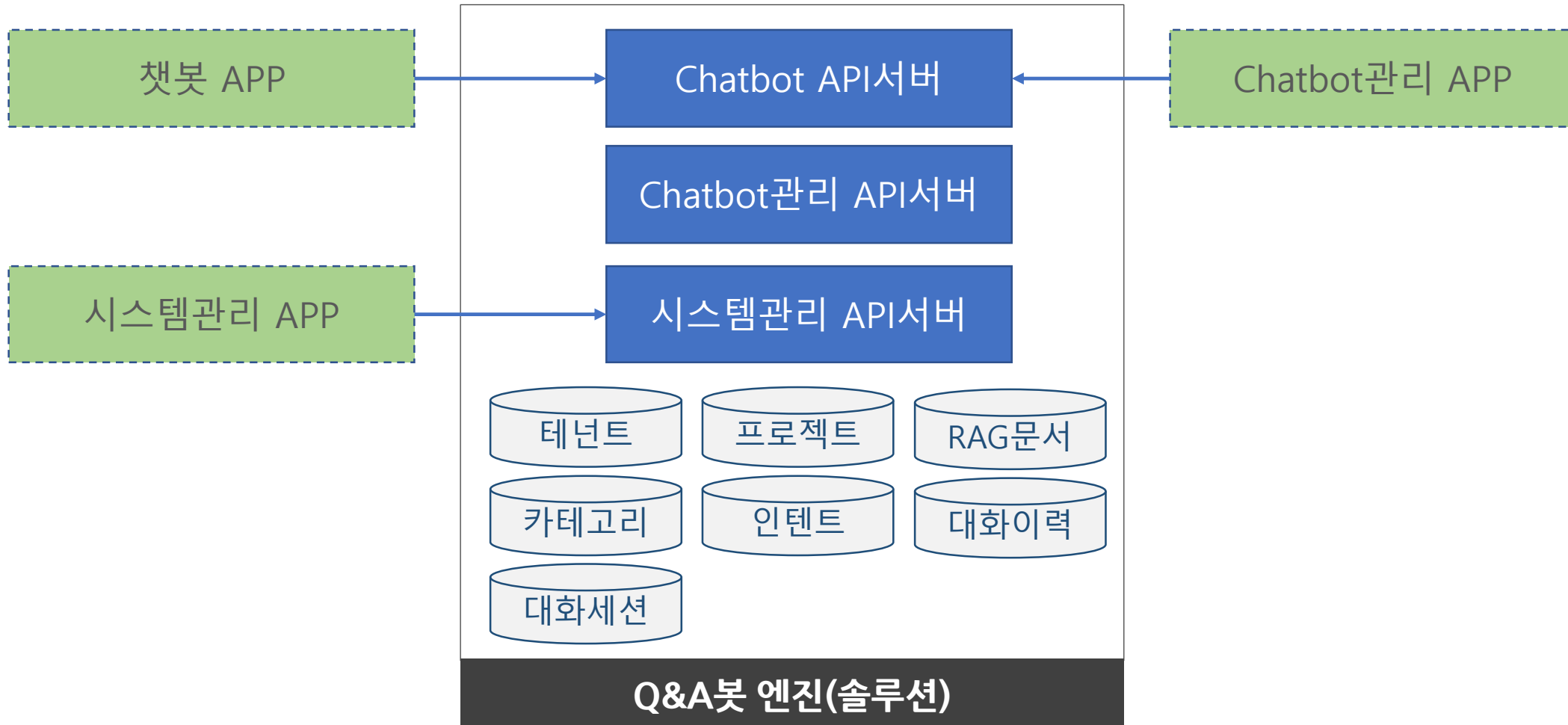
3.2 Q&A봇 구성

- 기본모드와 보정모드로 구성하여, 기본모드는 RAG 통한 문서(첨부파일, 웹, 텍스트) 수집 후 답변 가능
 - 보정모드는 RAG + 인텐트 등록 후 답변 가능
- 1) Chatbot API서버: 대화 추론, 대화 세션관리(세션ID생성, 대화시작, 대화종료)
 - 2) 시스템관리 API서버: 테넌트, 필요한 경우 사용자, 조직관리 추가 가능, 현재는 비식별ID만 관리
 - 민감정보를 솔루션에 저장 않는 전략
 - 3) Chatbot 관리 API서버: 프로젝트(봇), RAG문서, 인텐트 카테고리, 인텐트, 대화이력 및 오류신고

3. 제안

❖ Q&A봇 시스템 구성

* 본 연구의 범위는 Q&A봇 엔진(솔루션)으로 한정 함.



* Q&A봇 엔진(솔루션)

3. 제안 – 3.2 Q&A봇 구성 (1/4)

❖ Chatbot API서버 (1/2)

- 1) 개요: 대화추론, 대화세션관리, 답변오류신고 등의 기능으로 구성
 - UI 연동한다면 챗봇 사용자가 UI를 통해 챗팅과 답변오류신고 가능
- 2) 대화추론: 사용자의 질의 요청이 왔을때 기 등록된 정보 활용하여 적절한 답변 처리
 - 미리, 시스템관리API 통해 테넌트 DB 등록, Chatbot관리서버 API 통해 RAG문서, 인텐트 DB 등록
 - 대화추론API가 호출될때마다 대화내역을 대화이력 DB에 저장
 - 사용자 질문이 왔을 경우 인텐트 정보를 유사도 기반 검색 후 없으면 RAG기반 LLM 챗봇 호출
- 3) 대화세션관리
 - 세션ID생성, 대화시작, 대화종료 등의 API 호출을 통해 대화세션 DB에 정보 수정
 - 챗봇 대화 시작 / 종료 인식 등의 실질적인 세션관리는 호출하는 응용프로그램이 수행
 - 모니터링/통계/과금 등에 활용될 최소 정보만 솔루션에 저장

3. 제안 – 3.2 Q&A봇 구성 (2/4)

❖ Chatbot API서버 (2/2)

4) 답변오류신고

- 대화결과에 대해 오류가 있는 경우 답변오류신고 API를 호출
 - 대화추론시 대화내역이 대화이력 DB 저장
 - 대화이력 DB에서 해당 대화세션의 개별챗에 "오류신고여부" 값을 "Y"로 수정
 - 오류처리단계코드를 '01:최초오류등록' 으로 수정
- 이후 Chatbot관리 API서버의 "오류신고관리" 기능을 통해 오류상태를 관리

* 오류처리단계코드: 01: 최초오류등록 02:접수 03:처리중 04:조치완료

5) 유사문서검색모듈

- TFIDF벡터와 코사인 유사도를 통해 유사도 산정 방식을 기본으로 사용
- 사용자의 질문이 인입되면 RAG기반 LLM 보다 우선순위로 인텐트에 대해 유사문서검색 통해서 답변
- 코사인 유사도 기반 답변, 음절일치율 기능 확장 예정 (프로젝트 DB의 "인텐트유사도타입코드"로 분기)

3. 제안 – 3.2 Q&A봇 구성 (3/4)

❖ 시스템 관리 API서버

- 1) Chatbot 솔루션 제공 회사의 시스템 관리자 또는 고객사의 시스템 관리자가 사용함.
 - 2) 시스템관리는 시스템공통 기능을 제공하는 API서버로 고객사의 기본정보인 테넌트 정보 관리에 중점
 - 민감정보 분리위해 API마다 파라미터에 비식별ID 포함하며, 민감정보는 응용프로그램에서 관리
 - 필요할 경우 사용자정보, 조직정보 관리 가능하나, 테넌트 외의 정보는 응용프로그램에서 관리
- ※ 향후 민감정보 관리 전략에 대해 별도 연구 예정

3. 제안 – 3.2 Q&A봇 구성 (4/4)

❖ Chatbot관리 API 서버

- 1) 프로젝트(봇) 관리: 프로젝트(봇)의 전반적인 정보관리
 - 주요항목: LLM명코드, 임베딩모델코드, 벡터DB코드, 모드구분코드
- 2) RAG문서관리: RAG기반 LLM 서비스를 위해 문서/웹/텍스트 형태의 문서를 벡터DB에 등록
- 3) (인텐트) 카테고리관리: 인텐트가 많아질 경우 관리 편의를 위해 카테고리로 관리
 - ※ RAG문서와 인텐트에서 모두 카테고리 사용할수 있도록 설계 변경 예정
- 4) 인텐트관리: 유사문서탐색 통해 인텐트에 등록된 답변이 Chatbot 응답에 사용되도록 등록
- 5) 대화이력 및 오류신고관리
 - Chatbot 대화시 대화이력이 자동 등록
 - Chatbot 대화시 사용자는 잘못된 답변에 대한 오류신고 가능
 - 오류신고관리: 오류신고 접수 처리하고, 인텐트 매핑 후 Chatbot 정상 검색 되는 것 확인 후
오류신고 완료처리로 상태 변경하면 오류신고처리 완료 함.

3. 제안

3.3 유사문서 검색 모듈

- 1) TF-IDF벡터와 코사인 유사도 통한 질문과 인텐트내 질문의 유사도 비교
- 2) 유사도 가장 높은 질문이 속한 인텐트의 답변을 Chatbot 응답에 사용
- 3) 필수 엔티티와 불용어 인티티 이용 발화 의도 세분화

ex) 다른 성격의 인텐트 : “매트리스 교체시기 문의” , “정수기 교체시기 문의”

“맨트리스 교체시기 문의” 인텐트의 경우 “매트리스” 필수엔티티,

“정수기 ” 는 불용어 엔티티로 등록하여 의도 세분화하고, 인텐트 식별시 모호성 개선

- 4) 업무 성격에 맞게 적정 유사도 임계치 찾아 오인식이나 미인식 조절
 - 유사도 임계치 즉 유사도가 낮을수록 일치하는 단어 비율이 낮아도 추천한다.
 - 인텐트 구축 초반에는 유사도 임계치를 낮게 하여 최대한 많이 검색, 점차 임계치 높이는 전략
 - 오인식: 사용자의 발화 의도를 잘못판단하여 잘못된 답변으로 응답
 - 미인식: 사용자의 발화 의도를 찾지 못함.

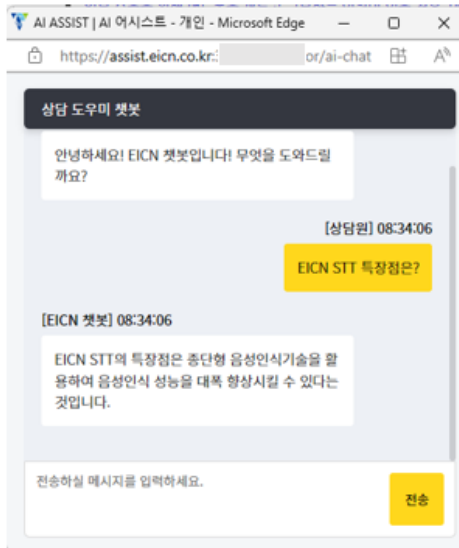
※ 인텐트내의 엔티티를 변수처리하는 슬롯필링 연구는 향후 대고객 Chatbot 및 콜봇 서비스에서 연구 예정

※ 프로젝트 DB의 "인텐트유사도타입코드"에 다른 유사도탐색 모듈을 추가하여 확장 분기처리 가능

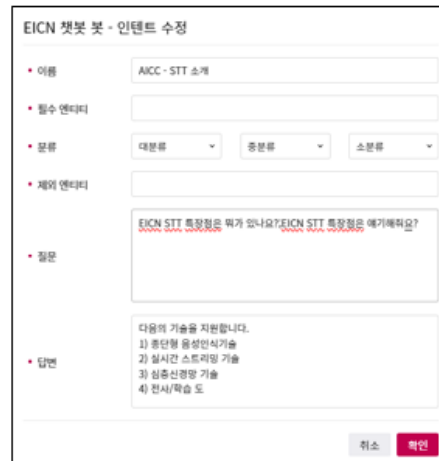
3. 제안

3.4 연동방안

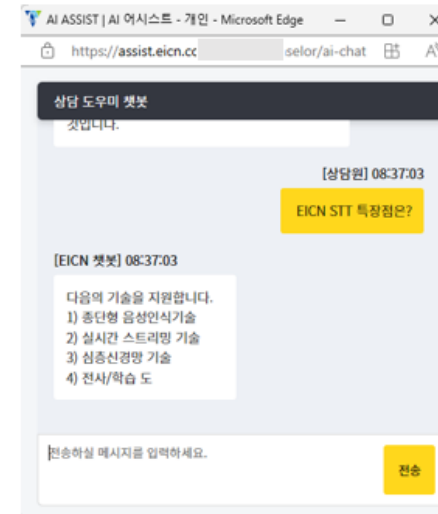
- 1) Chatbot API서버에서 유사문서 검색 공통모듈 먼저 호출 후 없으면 LLM 추론하는 방식으로 연동
- 2) 프로젝트(봇)가 보정모드이면서 인텐트에 고객 질문과 유사 또는 일치하는 질의-답변 셋이 등록 된 경우, 사용자의 질문이 들어오면 Chatbot API서버는 LLM 추론보다 유사문서 검색모듈을 우선 호출



a) 홈페이지 수집 내용 답변



b) 인텐트 등록



c) 인텐트 등록 후 답변

* 업무 Q&A 챗봇 연동 예)

4. 아키텍처 및 구현

❖ 개발방법론

- 1) 에자일방법론 : 요구사항 변동과 추가 기능 개발 요청이 있는 경우 비교적 확장 용이
- 2) 워터폴방법론: 폭포수방법론으로, 명확한 요구사항 중요, 전체 프로젝트에 대해 순차적 진행

❖ 아키텍처 및 구현

- 1) 요구사항 변동과 추가 기능개발 요청에 유연한 에자일방법론으로 개발
 - ※ 개발주기별로 각 스테이지에서 공통 문제 발견시 기존 해당 소스 변경 및 향후 보완방안으로 진행
- 1) 시스템 구성: Chatbot API서버, 시스템관리 API서버, Chatbot관리 API서버
- 2) 프로그램 호출구조
 - APP: API 호출 및 API input / out 담당
 - Biz : 전반적인 비즈니스 로직
 - 스키마: FastAPI의 Pydantic 기반의 클래스로 OpenAPI 스키마 개념이용, schema 접미사로 명명
 - 모델: SQLAlchemy기반의 데이터베이스 레이아웃 클래스로 model 접미사로 명명

4. 아키텍처 및 구현

❖ 4.1 기술 스택

* API개발자가 응용프로그램 및 모델서빙개발자와 입력/출력 레이아웃 협의

1) 다중 LLM: 기본 모델은 "gpt-3.5-turbo"로 구현, 모델별로 프로그램파일 분리

2) 다중 임베딩: OpenAI의 "text-embedding-3 모델"로 기본 개발

오픈소스 baai/bge-m3 모델 추가 예정

3) 다중 벡터DB: 기본 크로마DB로 구현, 향후 확장성을 위해 Milvus, 엘라스틱서치, 파인콘 고려

4) LLM 오케스트레이션 플랫폼: 기본 Langchain으로 구현, 그외 LLM모델 따라 적용

5) 웹 프레임워크: FastAPI

6) 트랜잭션방식 / 멀티프로세싱: FastAPI와 Uvicorn의 동기식 메소드로 일괄 개발

* 향후 사안에 따라 Gunicorn연계 또는 비동기 메소드로 변경

7) 공유자원에 대한 접근 제한 및 API호출 제한: 세마포어 등의 동시성 제어 (구현 예정)

8) 데이터베이스 처리 및 모델링: SQLAlchemy의 ORM(Object Relational Mapper)

4. 아키텍처 및 구현

❖ 4.2 응용프로그램 연동 구현 (1/3)

- 1) 구현단계에서 유연하고 확장가능한 아키텍처 기반 다양한 이해관계자 참여 개발
 - 솔루션 개발자(API개발자, 모델서빙 엔지니어), 영업담당자, 서비스 기획자, 응용개발자
- 2) 프로그램 사용자 편의성과 특정기능이 다른 서비스에 영향 주지 않는 것에 중점
- 3) 챗봇관리API서버의 API를 응용 프로그램에서 호출하여 구현
- 4) 구성: AI봇 (프로젝트) 설정, RAG문서관리, 카테고리, 인텐트관리

The screenshot shows a web interface for managing chatbots. At the top, there are tabs: 'AI 봇 설정' (selected), 'RAG 문서 관리', '카테고리', and '인텐트 관리'. To the right of the tabs are a toggle switch labeled '접기/펼치기', a '검색' button, and a '초기화' button. Below the tabs, there are two input fields: '봇 이름' with the value 'EICN' and '서비스 상태' with a dropdown menu showing '전체'. Below these fields, it says '전체 2 건' and there is a '추가' button. At the bottom, there is a table with 7 columns: 'No', 'AI 봇 이름', 'AI 봇 설명', '프로젝트 타입', '언어모델 타입', '언어모델', and '상태'.

No	AI 봇 이름	AI 봇 설명	프로젝트 타입	언어모델 타입	언어모델	상태
1	EICN 챗봇	EICN 챗봇	LLM 보정 모드	cloud	ChatGPT	활성

* Chatbot관리 메뉴 구성

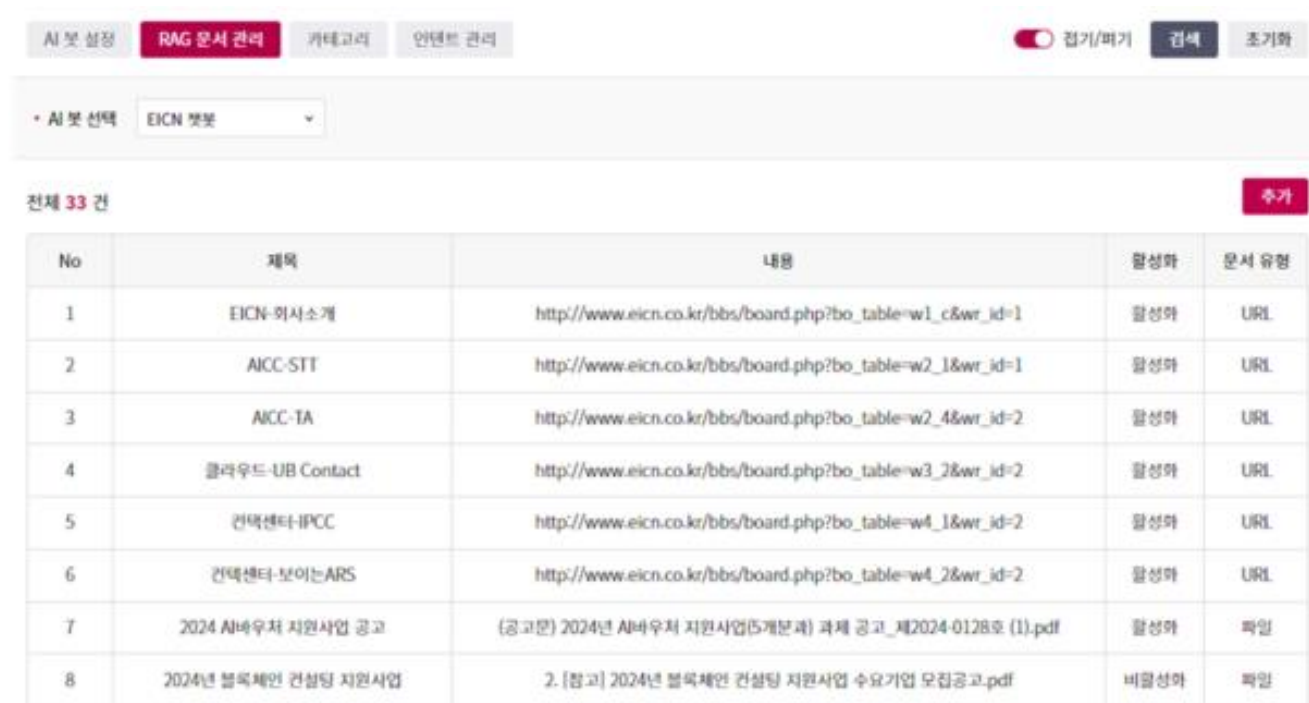
* 기존 인텐트용 카테고리에서 인텐트용과 RAG문서용 카테고리로 구분코드로 분리 설계 변경 중 25

4. 아키텍처 및 구현

❖ 4.2 응용프로그램 연동 구현 (2/3)

5) RAG 문서 관리 : 복수의 웹과 파일 업로드하여 벡터DB 저장

- 시스템 부하를 줄이기 위해 RAG대상 문서 업로드 후 적용버튼 클릭하여 벡터DB 저장 기획 및 구현
- 사용자가 이해하기 어렵고 불편한 프로세스라는 피드백 반영하여 동시 업로드 위해 설계 변경 중



No	제목	내용	활성화	문서 유형
1	EICN-회사소개	http://www.eicn.co.kr/bbs/board.php?bo_table=w1_c&wr_id=1	활성화	URL
2	AICC-STT	http://www.eicn.co.kr/bbs/board.php?bo_table=w2_1&wr_id=1	활성화	URL
3	AICC-TA	http://www.eicn.co.kr/bbs/board.php?bo_table=w2_4&wr_id=2	활성화	URL
4	클라우드-UB Contact	http://www.eicn.co.kr/bbs/board.php?bo_table=w3_2&wr_id=2	활성화	URL
5	컨택센터-IPCC	http://www.eicn.co.kr/bbs/board.php?bo_table=w4_1&wr_id=2	활성화	URL
6	컨택센터-모이는ARS	http://www.eicn.co.kr/bbs/board.php?bo_table=w4_2&wr_id=2	활성화	URL
7	2024 AI배우처 지원사업 공고	{공고문} 2024년 AI배우처 지원사업(5개분과) 과제 공고_제2024-0128호 (1).pdf	활성화	파일
8	2024년 불복채안 컨설팅 지원사업	2. [참고] 2024년 불복채안 컨설팅 지원사업 수요기업 모집공고.pdf	비활성화	파일

* RAG문서관리

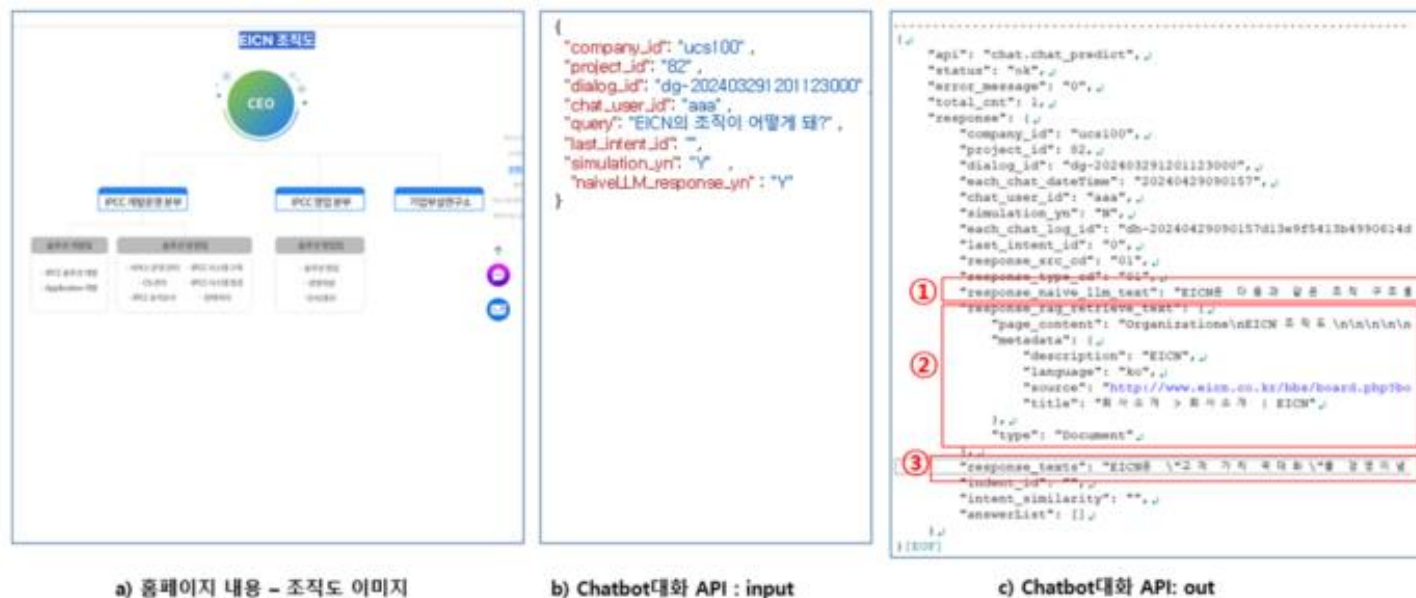
※ 응용프로그램에서 비동기 후처리로 업로드후 RAG저장 – 진행중 단계 추가하여 3단계 상태

4. 아키텍처 및 구현

❖ 4.2 응용프로그램 연동 구현 (3/3)

6) RAG기반 LLM 추론 + 유사문서검색 모듈

- Chatbot 핵심 기능: 벡터DB검색, LLM추론 유사문서검색기반 인텐트 식별 및 답변
- 잘못된 Chatbot 답변 원인 추적: c)-① 순수LLM답변, ② RAG검색결과, ③ RAG기반 LLM답변



* 잘못된 Chatbot 답변 원인 추적 예)

※ 위의 예서는 “EICN 조직” 문구 근처에 있는 회사소개” 글을 더 많이 검색, LLM도 잘못 답변

※ 3.4 연동방안의 Fig4와 같이 적합한 인텐트의 질문-답변 셋 등록 통해 보정

4. 결론

❖ 아키텍처

- 유연하고 확장 가능한 아키텍처
- 발전된 기술적용에 프로그램 구조 변경 최소화

❖ RAG문서등록 및 Chatbot

- 다양한 문서(첨부파일, 웹, 텍스트) 등록 후 즉시 Chatbot 통한 질의 / 답변

❖ 유사도 기반 Chatbot 연동

- 별도의 학습없이 인텐트 등록 후 답변 및 답변 오류 신고 및 오류 신고

❖ TF-IDF벡터 기반 코사인 유사도를 벡터DB로 대체 고려

- 도입하고자 하는 벡터DB가 전통 DBMS의 주된 기능 지원 여부 확인 필요

❖ 발전 방향

- 대고객용 Chatbot 시스템 서비스위한 멀티턴 Chatbot , 대화엔진을 콜봇에 확장 적용

5. References (1/2)

- [1] 신중민, 이재욱, 김경민, 이태민, 안성민, 박정배, 임희석 (2023), "QA Pair Passage RAG 기반 LLM 한국어 챗봇 서비스", 한국정보과학회언어공학연구회 2023년도 제35회 한글 및 한국어 정보처리 학술대회, 2023 Oct. 12, 2023년, pp.683 - 689
- [2] 황금하, 권오욱, 이경순, 김영길 (2017), "목적지향 대화 시스템을 위한 챗봇 연구", 정보처리학회논문지. KIPS transactions on software and data engineering. 소프트웨어 및 데이터 공학, v.6 no.11, 2017년, pp.499 - 506, <https://doi.org/10.3745/KTSDE.2017.6.11.499>
- [3] 정천수 (2023), LLM 애플리케이션 아키텍처를 활용한 생성형 AI 서비스 구현: RAG모델과 LangChain 프레임워크 기반 Generative AI service implementation using LLM application architecture: based on RAG model and LangChain framework 지능정보연구 = Journal of intelligence and information systems, v.29 no.4, pp.129 - 164, 삼성SDS AI Automation Team, <https://doi.org/10.13088/jiis.2023.29.4.129>
- [4] Yunfan Gao and Yun Xiong and Xinyu Gao and Kangxiang Jia and Jinliu Pan and Yuxi Bi and Yi Dai and Jiawei Sun and Meng Wang and Haofen Wang (2024), "Retrieval-Augmented Generation for Large Language Models: A Survey"
<https://doi.org/10.48550/arXiv.2312.10997>
- [5] Corrective Retrieval Augmented Generation(CRAG, 2024) <https://arxiv.org/abs/2401.15884>
- [6] Daniel Jurafsky & James H. Martin. 15. chatbots & Dialogue Systems, <https://web.stanford.edu/~jurafsky/slp3>
- [7] Rishi Bommasani (2022), "On the Opportunities and Risks of Foundation Model",
<https://doi.org/10.48550/arXiv.2108.07258>
- [8] 최남규, 민병철, 조우리, 민경은, 정한결 (2022), "한국어 오픈도메인 대화 시스템의 의인화 연구: 사회적 변인에 따른 상대높임법 중심", ACK 2022 학술발표대회 논문집, 29권 2호
- [9] 류명균 (2022), 인텐트 분류 모델의 신뢰도 임계값이 인간-에이전트 소통에 미치는 영향 연구 : 태스크 지향 음성 에이전트를 중심으로, 학위논문사항 학위논문(석사) 서울대학교 대학원, 2022.8 <https://www.riss.kr/link?id=T16548927>
- [10] 유현조, 송영숙, 김민수, 윤기현, 정유남 (2021), 딥러닝 기반 한국어 개체명 인식의 평가와 오류 분석 연구, 한국언어학회, vol.46, no.3, pp.803 - 828 DOI : <http://dx.doi.org/10.18855/lisoko.2021.46.3.010>
- [11] 서민택, 나승훈, 임준호, 김태형, 류휘정, 장두성 (2023), 검색 증강 LLM을 통한 한국어 질의응답, 한국정보과학회언어공학연구회, 제35회 한글 및 한국어 정보처리 학술대회,
- [12] 강대욱, 나승훈, 김태형, 류휘정, 장두성 (2022), "RAG를 이용한 한국어 오픈 도메인 질의 응답", 제 34회 한글 및 한국어 정보처리 학술대회

5. References (2/2)

- [13] LLM 비교: Quality, Speed, Price, <https://artificialanalysis.ai/models/mixtral-8x7b-instruct/providers>
- [14] Jianlv Chen and Shitao Xiao and Peitian Zhang and Kun Luo and Defu Lian and Zheng Liu, (2024), BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation, <https://doi.org/10.48550/arXiv.2402.03216>
- [15] FastAPI document, <https://fastapi.tiangolo.com/>
- [16] Gunicorn과 uvicorn 연계 Document, <https://www.uvicorn.org/#running-with-gunicorn>
- [17] asyncio.Semaphore document, <https://docs.python.org/ko/3.10/library/asyncio-sync.html>
- [18] SQLAlchemy document, <https://www.sqlalchemy.org/>
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela (2021), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, <https://arxiv.org/abs/2005.11401v4>
- [20] 김산, 이근배, (2023), LLM을 활용한 오픈 도메인 대화 시스템의 유해성을 완화하는 데이터 증강 기법, 한국정보과학회언어공학연구회, 제35회 한글 및 한국어 정보처리 학술대회, pp.346 - 351, <https://scienceon.kisti.re.kr/srch/selectPORSrchArticle.do?cn=NPAP14389423>